

Taxonomy

- **A taxonomy is a characterization of objects** or concepts based on the relationships that exist between them. The aim is **classification**.
- A taxonomy can be represented in a **hierarchical graph or table of attributes**, where each of the attributes identifies a particular element of the differentiation.
- The ESL taxonomy is composed of **five, not completely orthogonal, axes**.
 - Temporal
 - Data
 - Concurrency
 - Communication
 - Configurability

Temporal Axis I

- The temporal axis defines the **timing** in the model.
- The defined points are:
 - Partially ordered events
 - System event
 - Token cycle
 - Instruction cycle
 - Cycle-approximate
 - Cycle-accurate
 - Gate propagation accurate
- The **most abstract** point defined on the scale is **partially ordered events**. This means that we know when something will start and finish only in terms of its **relationship** to when **other things** start and finish in this particular execution. We cannot place any notion of actual times on these events.
- **Token cycle** accurate is typical in **dataflow systems** where a data arrival clock can be thought of as a **regularly scheduled event**.

Temporal Axis II

- The **instruction cycle**, **cycle-approximate**, and **cycle-accurate points** all have the notion of an **actual clock** with a **known period**, and thus actual times are known.
- At **instruction cycle accuracy**, **no account** is usually made of wait states or the impact that **hardware** may have on the execution times of software. The **memory accesses** normally happen only in a **virtual** sense.
- At **cycle-approximate** accuracy, the actual **operations** of the **bus or memory accesses** are present, but the timing between them is not known precisely.
- At the **cycle-accurate** point on the axis, **exact cycle counts** are known.
- **Gate propagation** accuracy is the final point on the axis at which the **timing within the clock period** is also known precisely.

Data Axis I

- The data axis defines the level of **precision of data**.
- The points on this axis are:
 - Token
 - Property
 - Value
 - Format
 - Bit Logical
- A **token** indicates that **some data is moving** through a system, but its **size and content are unknown**.
- A **property** may be an **enumerated type** where a name is given to something, even though how this name will be **represented is not defined**.
- **Value** could be an integer, floating-point value, or similar data type, where its **numerical accuracy is known** but how it is **represented in hardware is not known**.

- **Format** is a processor-like data format, dealing with issues such as endianness, and fixed- vs. floating-point.
- **Bit logical** provides that **mapping of value** into the **hardware** that will store it.

Concurrency Axis I

- **Concurrency** defines the amount of **processing** or execution of an application that can be performed **simultaneously**.
- An **implementation does not have to utilize all** of the available **concurrency** because this may produce a solution that does not fit the nonfunctional requirements of the design, such as cost, size, or power consumption constraints. In addition, **more concurrency** increases the **difficulty of verifying** an implementation.
- Points on this axis are:
 - Sequential (none)
 - Multi-application
 - Parallel
 - Multithread
 - Pipeline
 - Signal level

Concurrency Axis II

- **Sequential (none)** - No explicit concurrency exists in the function. This does not mean that it is not possible to extract concurrency from the function by looking at control or data dependencies, but that no indications exist in the function to identify such concurrency. An example of this would be ANSI C descriptions.
- **Multi-application** - The concurrency that exists here is very **coarse grained** and **not built into the operating characteristics of the system** directly. In other words, two independent functions may share some data contained in a file, but **one function has no direct influence on the other**. An example of this would be a Microsoft Word document and an Excel spreadsheet, where it is possible to embed data or charts from the spreadsheet into the document. Whenever the document is opened, the updated chart or data from the spreadsheet would be imported.

Concurrency Axis III

- **Parallel** - This level of concurrency concerns **multiple functions that are operating independently, but may need to cooperate for certain activities**. An example is a transaction processing system where data is explicitly shared between each function or instance of the function. Data locking on those accesses has a direct impact on the running of each function and may trigger further functions to be executed. Normally, one would not be able to predict when such synchronization points would happen because the events that initiate them are independent.
- **Multithread** - A single function may have been defined with multiple threads of execution. **This concurrency is thus explicitly built into the operation of the function and may have explicitly defined synchronization points**. The means of synchronization and data transfer are built into the function. Examples are: the operating

Concurrency Axis IV

system running on a computer, multiple pieces of a hardware system that have divided up the task into independent functioning blocks.

- **Pipeline** - In general, pipelining defines the **streaming of data from one operation to the next in a controlled manner**. There is a range of different granularities when we talk about pipelines. At **the coarsest level**, there could be large blocks of computation that could be executed within a dedicated processor, programmable coprocessor, or fixed-function custom hardware block, and the results of this may be fed to another, similar subsystem. At a **finer level**, pipelining is very common in processors. It could also be considered to be the default level of definition for an RTL description.

- **Signal level** - This is a very fine level of concurrency that is typical in a gate-level or asynchronous description. **Multiple combinatorial paths exist between two synchronization points** and signal transitions progress through the system as fast as the circuitry will allow. This may create transient values along the processing chain and would normally be combined with some degree of pipelined design so that synchronization points can be defined.

Communication Axis I

- When **more than one processing element** exists, there must be **communication** between them. That communication can take many forms and is highly dependent on the architecture of the final solution.
- The points on this axis are:
 - Point-to-point
 - Buffered
 - Coprocessor
 - Memory
 - Bus based (high speed)
 - Bus based (low speed)
 - None
- **Point-to-point** - This allows two concurrent functions to communicate with each other **directly without any additional form of control** between them. This would generally be found only in hardware solutions because it implies the continuous flow of information rather than something that is intermittent, periodic or not.

Communication Axis II

- **Buffered** (also includes FIFO) - This is the most **elemental** form of **controlled communications** that can be made between two concurrent tasks. It allows for the **execution rates of two functions to be different** and thus provides some degree of electrical and timing isolation between them. If the buffer depth is greater than one, as would be the case with a FIFO, then rate independence can also possibly be assured, at least to the depth of the FIFO.
- **Coprocessor** - Although this may sound like a processor-dependent term, it means that the **information processed across the two functions is being shared** in some manner. It is likely that one of the functions is the **owner of the information** and may have to do some work in order to make it available to the other function. In other words, a **producer–consumer relationship** exists between them. In the case of a processor/coprocessor pair, the processor would own the information, but at the time of the request the information may be in

Communication Axis III

a register, cache, or memory. The coprocessor is unaware of its actual location, but it does affect the rate at which the information is returned. A coprocessor can be programmable or fixed-function hardware.

- **Memory** - Memory is a very common way to transfer information between two functions. At this level, it is assumed to be **multiport** such that each function can read or write to the memory **independently**. It would also imply that they have a **dedicated connection** to the memory and not have to access it through a bus mechanism (see the next point on the scale). Examples of this often occur in a client–server type of relationship, where multiple clients may look for transactions placed in the memory that they are able to perform and extract the necessary data. This is unlike the buffered form of communications, where the data is ordered and transferred point to point.

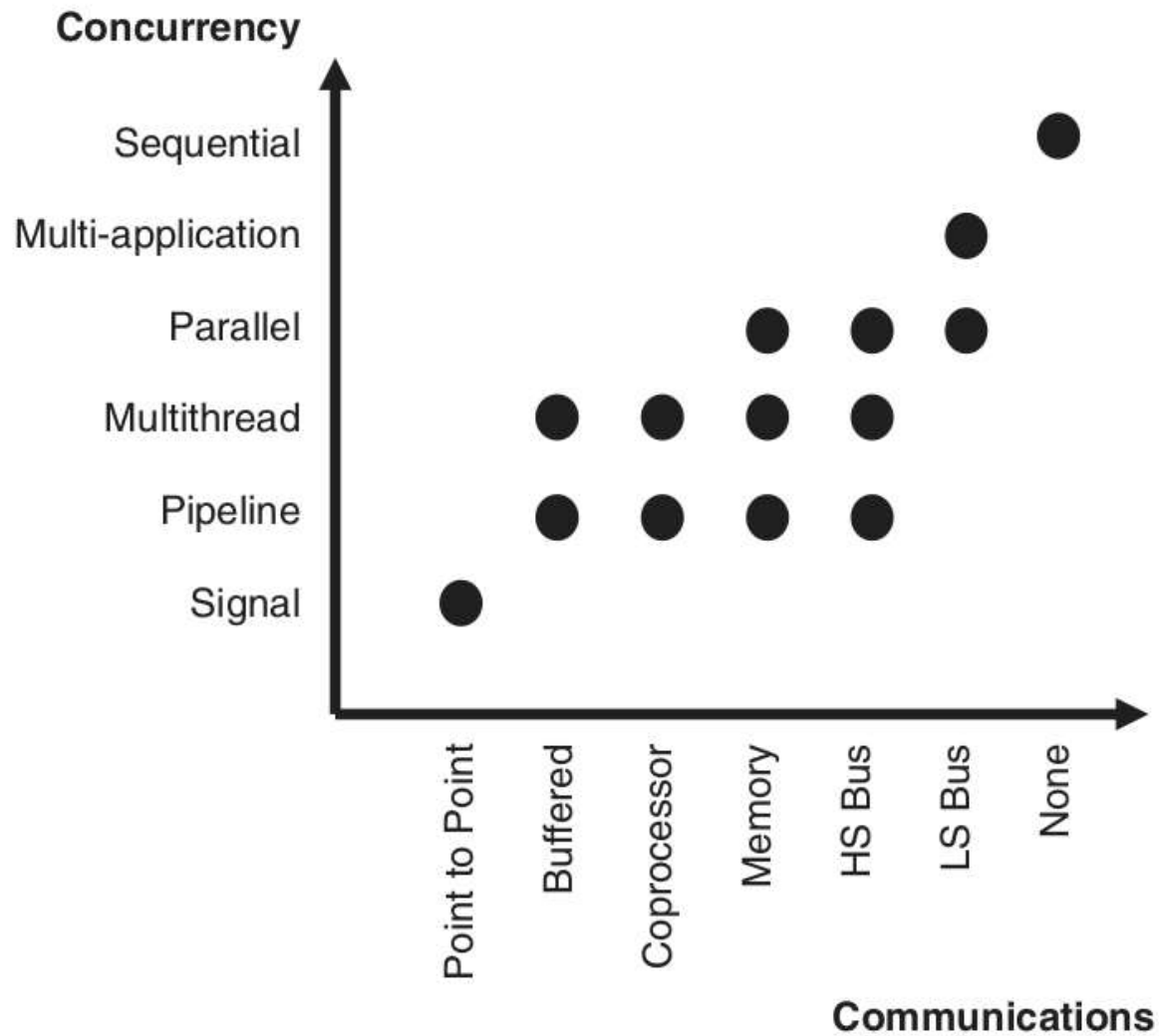
Communication Axis IV

- **Bus based (high speed)** - In software systems where the function is run on a processor, almost **all communications are made through one or more buses**, each of which may have different transfer rates or latencies. Several buses may be used to make the necessary transfer, with bus bridges mapping the requests between them. This category is meant to include all buses directly associated with the processor and does not, for example, make the distinction between the Advanced Microcontroller Bus Architecture (AMBA), Advanced eXtensible Interface (AXI), and AMBA Advanced Peripheral Bus (APB) buses of an ARM processor. This category would also contain the emerging architectures called Network On Chip (NoC). When a communication request is made between any two of the processors, the routing resources within each of the processing elements act as a bridge between each of the processor buses.

Communication Axis V

- **Bus based (low speed)** - In this context, a low-speed bus means that there is a more **extensive protocol** being used on the bus to ensure transport integrity, or that is capable of allowing things to be connected at much larger distances. Examples of this within a computer world would be a Peripheral Component Interconnect (PCI) bus or a Universal Serial Bus (USB). It could also include an Internet connection where functions would be communicating over great distances and have very long latencies.
- **None** - If the functions do not need to communicate, then no channel is necessary for their communication. This also implies that the concurrency between the functions does not create any issues with which the system designer must be concerned.

Concurrency and Communications



Configuration Axis I

- This axis describes the **configurability** of the solution.
- The points on this axis are:
 - Fixed
 - Design
 - Configurable
 - Programable
 - Dynamically reprogrammable
- **Fixed** - The user of a fixed block or device has **no ability to make any kind of changes** to it. A fixed design may be provided at any level of abstraction, in hardware or software. It may be provided as a black box lacking any kind of visibility into its internals, although this does not have to be the case. A **compiled object file** or executable is an example of a fixed design in software.

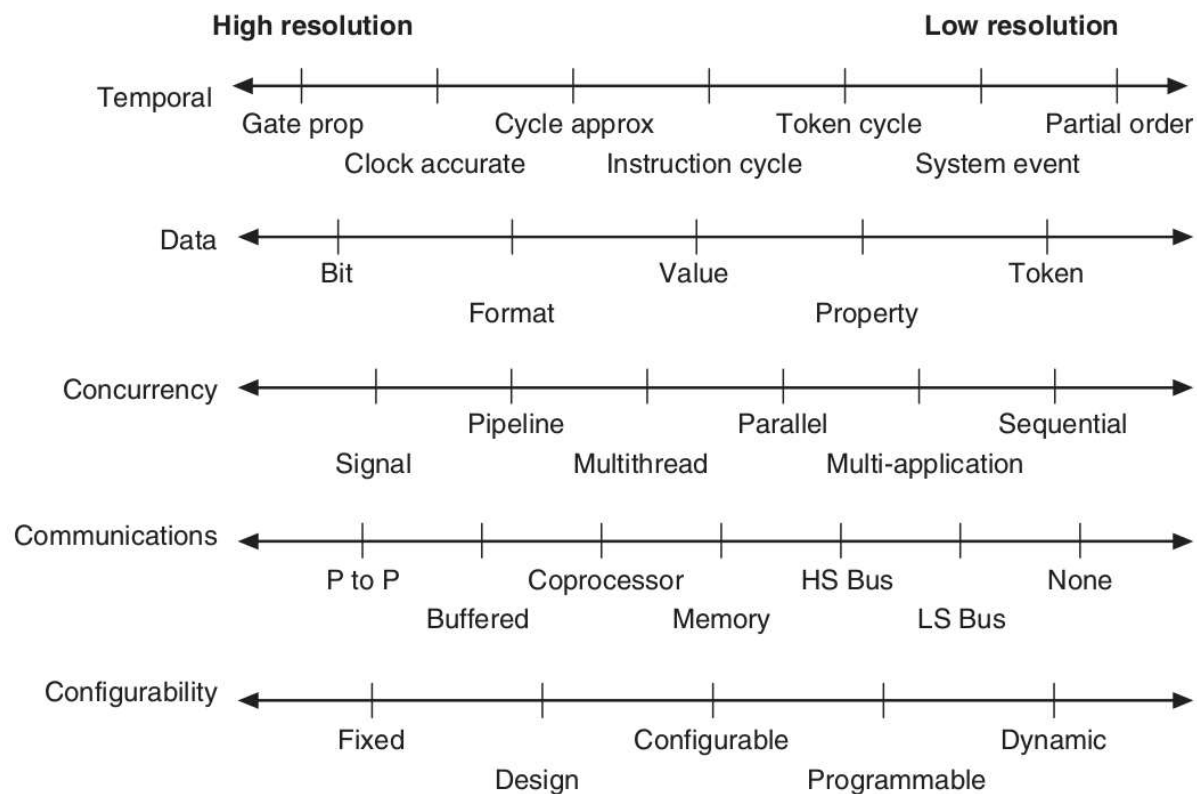
Configuration Axis II

- **Design** - Configurability at the design level is possible for any design where the **original source is available**. Almost all designs are configurable at design time. This is considered the normal design flow for hardware and software components.
- **Configurable** - **Configurable IP** blocks come with a set of **configuration options**. These blocks can be personalized to take on many different roles. Examples would be a Universal Asynchronous Receiver/Transmitter (UART) or a communications processor with options to control the protocol to be used. Once these options are set, they cannot be modified during operation; neither can additional configuration options be added without redesign.

Configuration Axis III

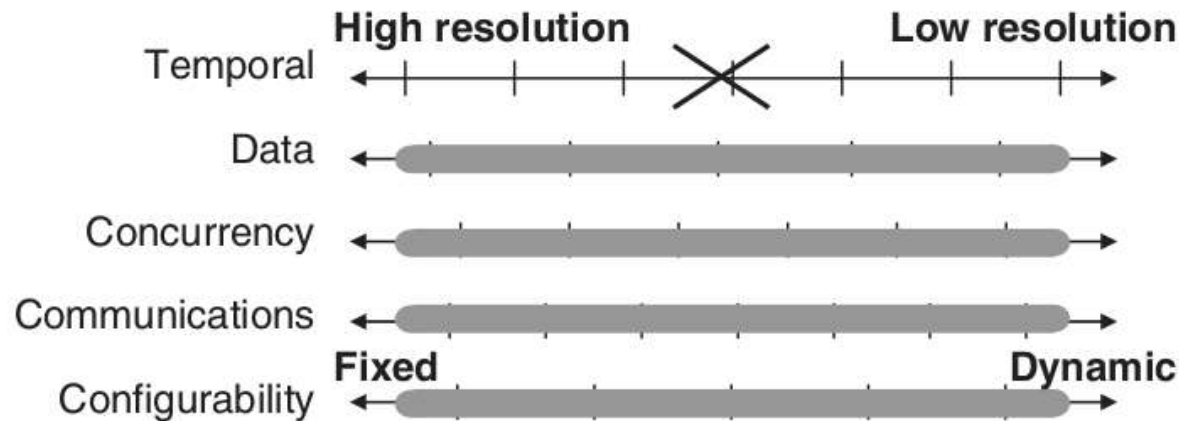
- **Programmable - Programmable devices** such as FPGAs and PLDs can take on any number of personalities by loading configuration information at start-up time. New configurations can be defined over time. Because of their **slow configuration** times, they are not considered to be dynamically reprogrammable.
- **Dynamically reprogrammable** - These devices may be reconfigured with new personalities by **loading configuration data dynamically during operation**. Dynamically reconfigurable devices can be treated as a resource in the system that can be scheduled and programmed to perform any number of functions. Some types of FPGAs are dynamically reconfigurable.
- This axis is **orthogonal to concurrency and communication** axis.

ESL Taxonomy Axis



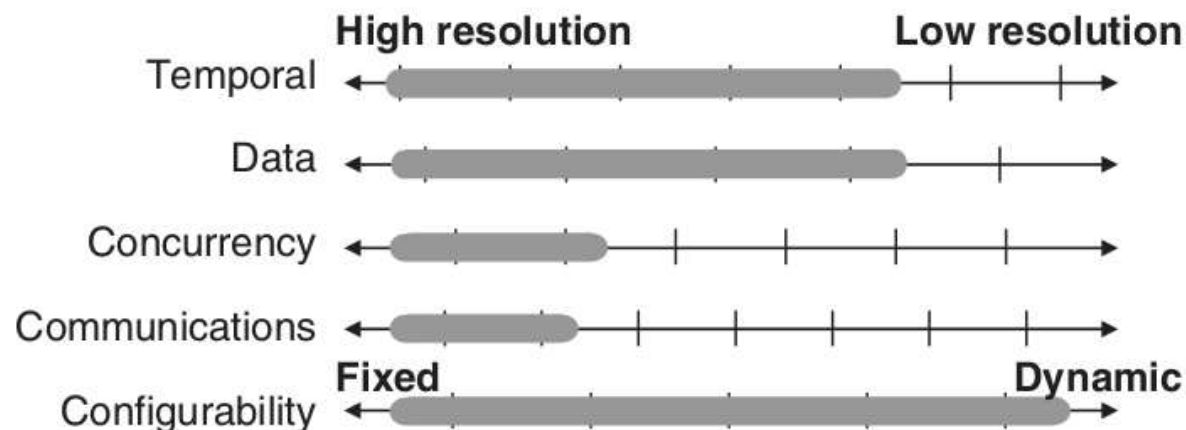
Example - Functional model

- A functional model defines the operations of a system **without describing a particular implementation**. Functional models can be written at any level of abstraction and can have any of the ESL attributes. The major characteristic of a functional model is that it does **not define timing**.



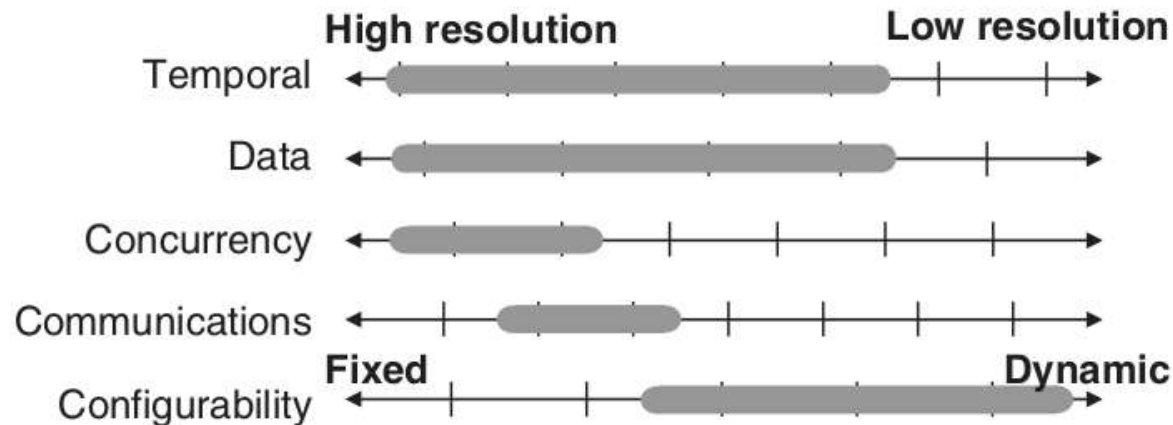
Example - HDL Description

- A pure **hardware solution** written in **Verilog** or **VHDL** is most likely to use **point-to-point** or **buffered communication** with tight communication between each of the elements. It is possible to define complete systems with multiple processors using these languages, but in this capacity they are being used as the structural interconnect for building blocks probably defined at higher levels of abstraction. The intent here is the type of input that could be **fed into a synthesis tool** to produce hardware.



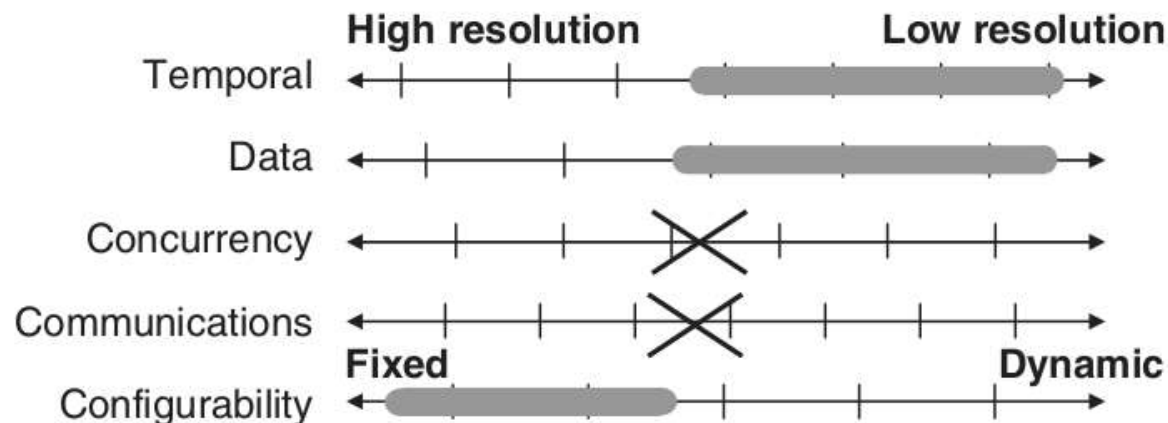
Example - FPGA Solution

- If we were to consider a subset of designs targeted at an FPGA, then configurability is performed either at start-up or dynamically. Inside the FPGA, communication may either be through **signals**, although this is limited because of timing issues, through buffers, or through local memory. Concurrency is predominantly fine grained.



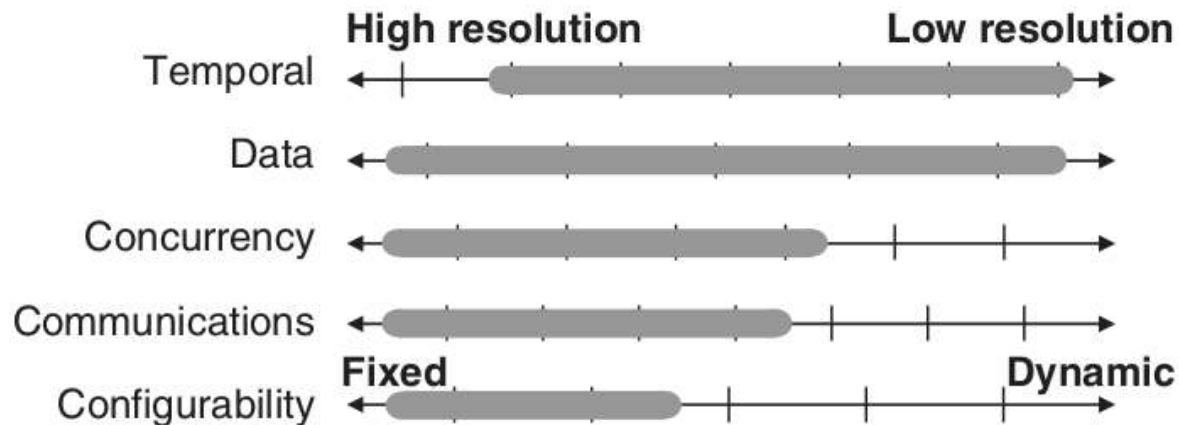
Example - Generic C Solution

- The C language has **no built-in** constructs for dealing with **concurrency or communication**. These are capabilities provided by an operating system. Thus, the C language is a highly restricted entry language for concurrent systems. This has led to the creation of many variants of it to deal specifically with concurrency. C has a **low resolution** when it comes to **data** representation, with a variable having a defined value being the closest it can get to hardware. It also has **no built-in** language constructs for the management of **time**. **Configurability is at the design level** because C and C ++ code is not considered to be changeable once it has been compiled and shipped



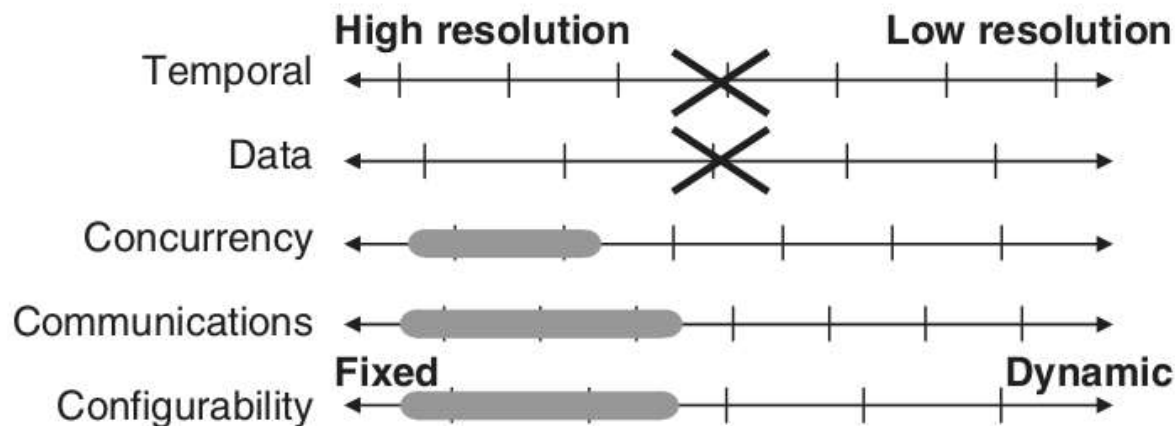
Example - SystemC Solution

- SystemC is popular C ++ derivative. This language **adds concurrency** and several explicit means of **communication**, including signal-based and transaction-based as well as messaging through shared variables.



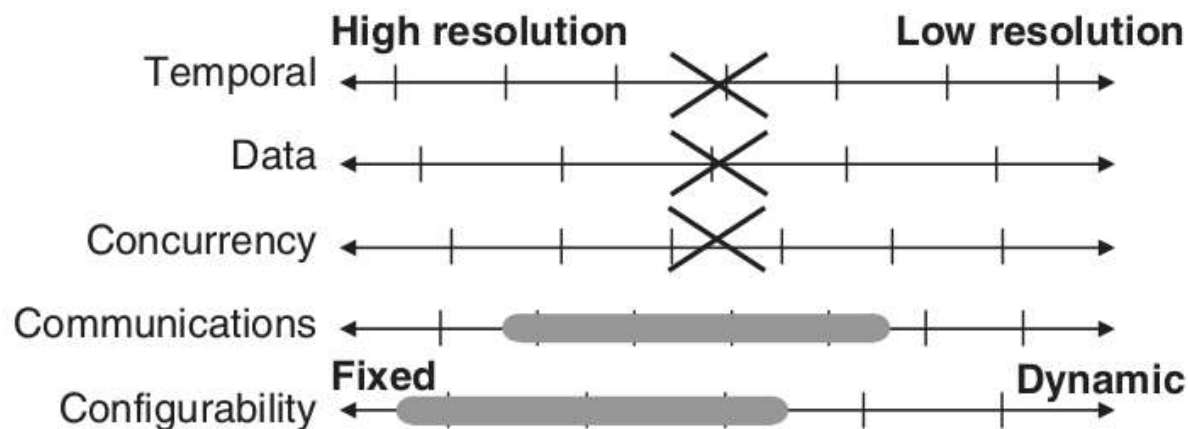
Example - Extensible Processor Solutions

- Many of the tools available today focus on partitioning functionality between a processor and one or more blocks of hardware that are automatically created. Hardware is used to implement **part of the functionality** that consumes a **large percentage of the execution time**. There are a number of ways in which **this logic can be attached to the processor**.
- In extensible processors, there is a very close coupling where the **added hardware becomes an instruction** in the processor itself, directly sharing resources with other instructions.



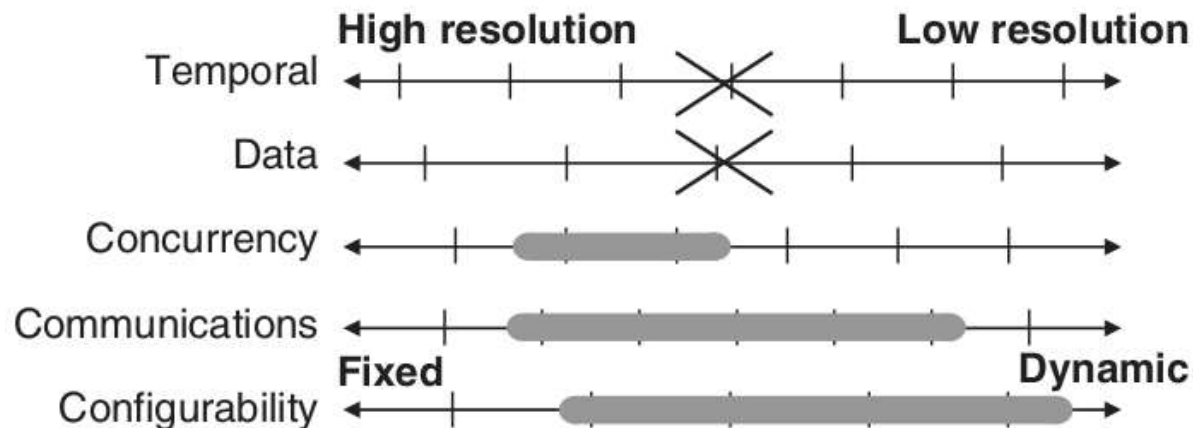
Example - Processor plus fixed-function coprocessor

- Also common are solutions where a coprocessor is created. It is **common** in coprocessor solutions for the functionality performed in the processor and the **functionality performed in the coprocessor to operate serially**, without achieving a great degree of concurrency. Such solutions look at speeding up part of the problem so that it has a significant impact on the total execution time **without necessarily increasing concurrency**. Alternatively, by reducing total execution time, it may become possible to slow down the main processor so that the gain is translated into lower power.



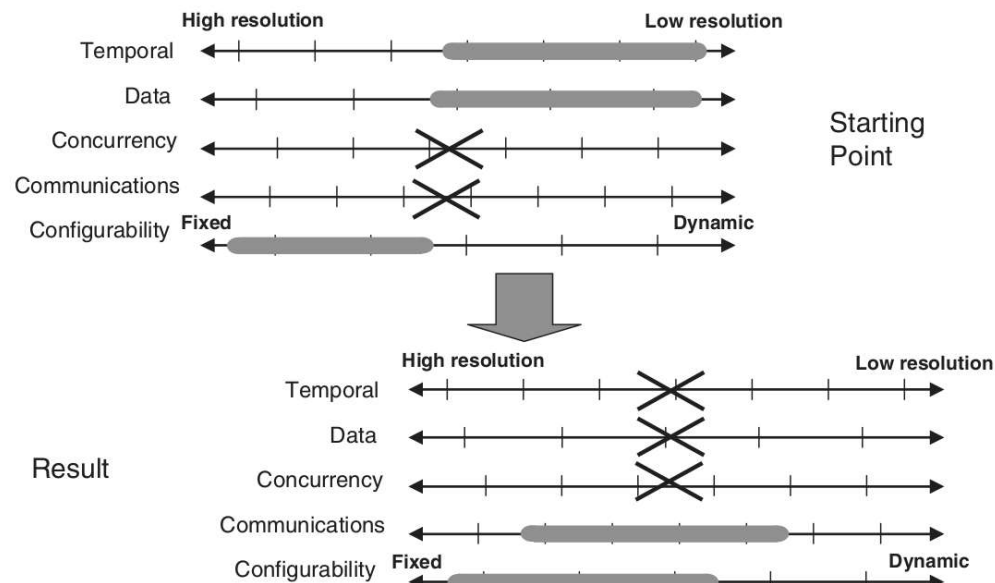
Processor plus programmable coprocessor

- Over time, it is likely that these solutions will improve their capability for **taking advantage of concurrency**. It is also possible that **coprocessors** will be generated that are capable of **more than one function**, giving them **greater configurability**.



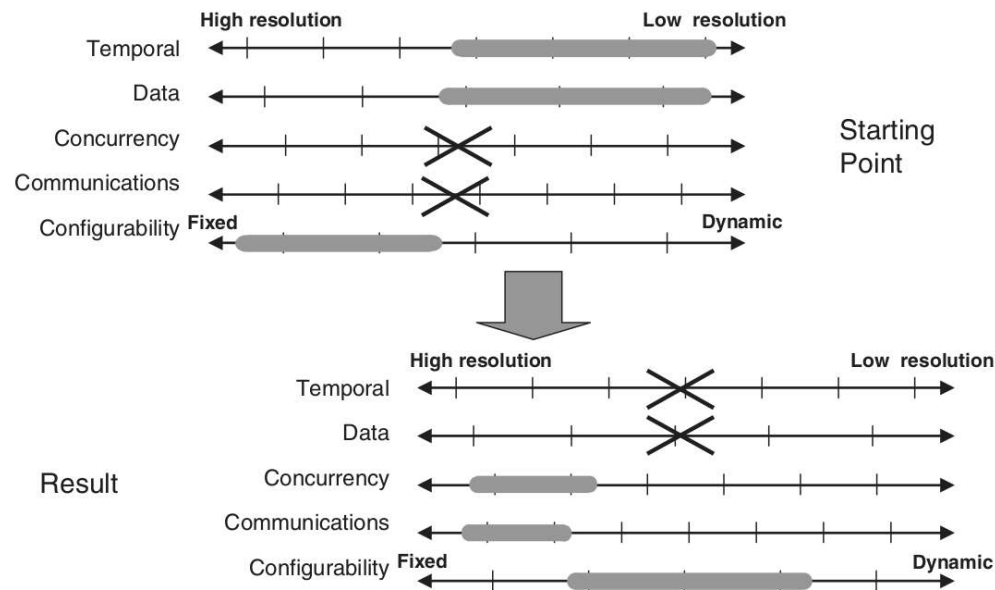
Example Design Flow - 1

- Poseidon Design
- This tool **starts from a generic C description** and performs analysis on the code, identifying the parts of it that can be migrated from a general-purpose processor **onto a highly targeted coprocessor**. The solution has the ability to target different levels of communication between the processors, such as memory-based or a direct interface into the processor. The solution shown **does not add concurrency** - in other words, the main **processor is stopped** when the coprocessor is running.



Example Design Flow - 2

- Poseidon Design - pipelined
- The **coprocessor** created by same tool can have a **pipelined architecture**, and thus adds concurrency within this processor. It is also possible that multiple functions could be implemented with the coprocessor, with the infrastructure being shared among them.



Example Design Flow - 3

- Forte
- This tool start from a **SystemC** description and perform what could be described as more classic **high-level synthesis** with the aim of producing hardware that can implement the desired functionality.

